

UNIVERSIDAD POLITÉCNICA TERRITORIAL DE FALCÓN “ALONSO GAMERO”

PROGRAMA NACIONAL DE FORMACIÓN: INFORMÁTICA UNIDAD CURRICULAR: ALGORÍTMICA Y PROGRAMACIÓN

DOCENTE: LCDO. CARLOS NOGUERA
UNIDAD I: ALGORITMOS Y PROGRAMAS

FORMATO PARA LA IMPLEMENTACIÓN DE ALGORITMOS:

Enunciado: En el contexto actual de la informática, los usuarios a menudo dependen de múltiples aplicaciones pesadas y separadas (una app de calculadora, consultas de divisas en navegadores web, juegos casuales independientes) para realizar tareas cotidianas sencillas. Esto genera tres problemas principales, consumo de recursos, pérdida de foco y tiempo (Context Switching) y Fragmentación Educativa.

1. ANÁLISIS

Descripción de la solución: Este Mini Sistema Operativo CLI resuelve estas problemáticas ofreciendo un entorno unificado, ultra-ligero y libre de distracciones. Al centralizar herramientas vitales (matemáticas, financieras) y opciones de esparcimiento (juegos) en una sola interfaz de texto, el programa consume una cantidad ínfima de recursos del sistema anfitrión, respondiendo de manera instantánea.

Entradas (Inputs)	Salidas (Outputs)
Navegación del Sistema: Selección de opciones a través de números enteros (del 1 al máximo de opciones disponibles en el menú principal). Datos de Aplicaciones Utilitarias: * Calculadora Científica: Ingreso de expresiones matemáticas o números y operadores. Conversor de Divisas: Ingreso del monto a convertir y selección de la moneda de origen y destino. Datos de Entretenimiento (Juegos): Adivina el Número: Ingreso de intentos numéricos enteros. Piedra, Papel o Tijera: Selección numérica o de texto (strings) representando la jugada del usuario.	Interfaz Gráfica de Texto (TUI): Menús interactivos delineados con separadores visuales (=====) que muestran las aplicaciones instaladas: Calculadora Científica Conversor de Divisas Juego: Adivina el Número Juego: Piedra, Papel o Tijera Manual de Uso Información del SO y Créditos Salir Resultados Computados: Valores numéricos con formato exacto o decimal (resultados matemáticos y conversiones monetarias). Feedback de Usuario: Mensajes de estado como alertas de error ("Entrada

Señales de interrupción: Teclas como [ENTER] para continuar o comandos de salida.	inválida, intente de nuevo"), pistas interactivas de los juegos ("¡Estás muy caliente!") y notificaciones del sistema ("Saliendo del OS...").
Proceso	
Expresión Matemática	Expresión Algorítmica
Conversión Monetaria en Tiempo Real Formula: $\text{Monto_Final} = \text{Monto_Original} * \text{Tasa_Cambio}$ Donde: - Monto_Final: Cantidad de dinero ya convertida a la moneda de destino. - Monto_Original: Cantidad de dinero ingresada por el usuario en la moneda base. - Tasa_Cambio: Valor de cambio entre ambas monedas obtenido desde la API.	En Python (Convertidor_divisa.py): <pre>resultado = monto * tasa_cambio</pre> En PSeInt (Reglas Básicas): <pre>total <- monto * tasa</pre>
Cálculo del Tiempo Transcurrido en Animaciones Formula: $\text{Tiempo_Transcurrido} = \text{Tiempo_Total} - \text{Tiempo_Restante}$	En Python (utils.py): <pre>elapsed = seconds - remaining</pre> En PSeInt (Reglas Básicas):

Donde: - Tiempo_Transcurrido: Segundos que ya han pasado durante la barra de carga. - Tiempo_Total: Segundos totales de espera configurados en la función. - Tiempo_Restante: Segundos que faltan para terminar la animación.	elapsed <- seconds - remaining
Ajuste de Opción del Menú Principal Formula: Indice_Lista = Opcion_Usuario - 1 Donde: - Indice_Lista: Posición real en memoria (Base 0) para ejecutar la función interna. - Opcion_Usuario: Número entero de la opción (Base 1) que el usuario ingresa por teclado.	En Python (main.py): opcion_usuario = int(input()) - 1 En PSeInt (Reglas Básicas): opcion_usuario <- opcion_usuario - 1

2. DISEÑO

Pseudocódigo

Descripción de los pasos para llevar a cabo la solución del problema.

```
// Algoritmo Principal: Simulador de Sistema Operativo UPTAG CLI OS
// Asignatura: Algoritmo y Programación 1 – Profesor Carlos Noguera
// Trayecto I – Sección: UPTAG PNfi S10 T1T1.
Algoritmo UPTAG_CLI_OS
Definir usuario Como Cadena
Definir opcion_seleccionada Como Entero
```

```

Definir sistema_encendido Como Logico
Definir pausa Como Cadena

Escribir "||| ATENCION: ESTA VERSION ES UN PROTOTIPO DE PSEUDOCODIGO DE USO TECNICO Y NO
APTO PARA COMERCIALIZAR. Version oficial completa:
https://github.com/CiszukoAntony/uptagcli/releases ||"

Escribir "Iniciando UPTAG CLI OS..."

// Inicialización con parámetros (este sí lleva paréntesis porque tiene argumento)
init_username(usuario)

sistema_encendido <- Verdadero

Mientras sistema_encendido Hacer
  Borrar Pantalla
  Escribir "=== MENU PRINCIPAL ==="
  Escribir "Usuario actual: ", usuario
  Escribir "1. Calculadora Científica"
  Escribir "2. Conversor de Divisas"
  Escribir "3. Juego: Adivina el Numero"
  Escribir "4. Juego: Piedra, Papel o Tijera"
  Escribir "5. Manual de Uso"
  Escribir "6. Informacion del SO y Creditos"
  Escribir "7. Apagar Sistema"
  Escribir "===== "
  Escribir "Seleccione una opcion (1-7):"
  Leer opcion_seleccionada

  Segun opcion_seleccionada Hacer
    1:
      Borrar Pantalla
      calculadora_cientifica
    2:
      Borrar Pantalla
      conversor_divisas
    3:
      Borrar Pantalla
      juego_adivina_numero
    4:
      Borrar Pantalla
      juego_piedra_papel_tijera
    5:
      Borrar Pantalla
      manual_uso
    6:
      Borrar Pantalla
      informacion_so
    7:
      Borrar Pantalla
      Escribir "Saliendo del sistema operativo..."
      Escribir "Apagando..."
      Escribir "Gracias por usar UPTAG CLI OS."
      sistema_encendido <- Falso
      De Otro Modo:
        Escribir "Error: Opcion invalida. Intente de nuevo."
      FinSegun

  // Pausa de control universal compatible con cualquier perfil
  Si sistema_encendido Entonces
    Escribir ""
    Escribir "Presione ENTER para volver al MENU PRINCIPAL..."
    Leer pausa
    FinSi
  FinMientras
FinAlgoritmo

// =====
// MÓDULO CENTRAL: UTILS & CAPA DE CONTROL

```

```
// =====

SubProceso init_username(usuario Por Referencia)
Definir entrada Como Cadena
entrada <- ""
Mientras entrada = "" Hacer
Escribir "Cual es tu nombre?"
Leer entrada

Si entrada = "" Entonces
Escribir "Error: El nombre no puede estar vacio. Intente de nuevo."
FinSi
FinMientras
usuario <- entrada
FinSubProceso

SubProceso module_error(nombre, ubicacion, paquete, doc)
Escribir "Error: no estas ejecutando main, estas ejecutando el modulo directamente."
Escribir "__name__": ", nombre
Escribir "__file__": ", ubicacion
Escribir "__package__": ", paquete
Escribir "__doc__": ", doc
FinSubProceso

SubProceso main_error(nombre, ubicacion, paquete, doc)
Escribir "Error: hubo un error ejecutando el modulo principal."
Escribir "__name__": ", nombre
Escribir "__file__": ", ubicacion
Escribir "__package__": ", paquete
Escribir "__doc__": ", doc
FinSubProceso

// =====
// MÓDULO 1: CALCULADORA CIENTÍFICA (REGLAS BÁSICAS)
// =====
SubProceso calculadora_cientifica
Definir op Como Entero
Definir n1, n2, res Como Real

Escribir "--- CALCULADORA ---"
Escribir "1. Sumar"
Escribir "2. Restar"
Escribir "3. Multiplicar"
Escribir "4. Dividir"
Escribir "Seleccione operacion:"
Leer op

Escribir "Ingrese primer numero:"
Leer n1
Escribir "Ingrese segundo numero:"
Leer n2

Segun op Hacer
1:
res <- n1 + n2
Escribir "Resultado Suma: ", res
2:
res <- n1 - n2
Escribir "Resultado Resta: ", res
3:
res <- n1 * n2
Escribir "Resultado Multiplicacion: ", res
4:
Si n2 <> 0 Entonces
res <- n1 / n2
Escribir "Resultado Division: ", res
SiNo
Escribir "Error: No se permite la division entre cero."
FinSi
FinSubProceso
```

```

De Otro Modo:
Escribir "Error: Operacion no valida."
FinSegun
FinSubProceso

// =====
// MÓDULO 2: CONVERSION DE DIVISAS (CORREGIDO MULTILÍNEA)
// =====
SubProceso conversor_divisas
Definir desde_m, hacia_m Como Cadena
Definir monto, tasa, total Como Real

Escribir "--- CONVERSION DE DIVISAS ---"
Escribir "Monedas validas: USD, EUR, VES"

Escribir "Convertir desde (ej. USD):"
Leer desde_m
Escribir "Convertir a (ej. EUR):"
Leer hacia_m
Escribir "Ingrese el monto a cambiar:"
Leer monto
Si monto <= 0 Entonces
Escribir "Error: El monto debe ser un numero positivo mayor a cero."
SiNo
tasa <- 1.0

// Estructuras condicionales formateadas correctamente línea por línea
Si desde_m = "USD" Y hacia_m = "EUR" Entonces
tasa <- 0.92
FinSi

Si desde_m = "EUR" Y hacia_m = "USD" Entonces
tasa <- 1.08
FinSi

Si desde_m = "USD" Y hacia_m = "VES" Entonces
tasa <- 36.50
FinSi

Si desde_m = "VES" Y hacia_m = "USD" Entonces
tasa <- 0.027
FinSi

total <- monto * tasa

Escribir "===== RESULTADO ====="
Escribir monto, " ", desde_m, " equivalen a ", total, " ", hacia_m
Escribir "===== "
FinSi
FinSubProceso

// =====
// MÓDULO 3: JUEGO ADIVINA EL NÚMERO
// =====
SubProceso juego_adivina_numero
Definir secreto, num_usuario, conteo Como Entero
secreto <- Aleatorio(1, 20)
conteo <- 0
num_usuario <- 0

Escribir "--- JUEGO: ADIVINA EL NUMERO ---"
Escribir "He generado un numero del 1 al 20. Trata de descubrirlo."

Mientras num_usuario <> secreto Hacer
Escribir "Introduce tu numero:"
Leer num_usuario
conteo <- conteo + 1

```

```

Si num_usuario < secreto Entonces
Escribir "El numero secreto es mayor."
SiNo
Si num_usuario > secreto Entonces
Escribir "El numero secreto es menor."
FinSi
FinSi
FinMientras

Escribir "Correcto! Adivinaste el numero en ", conteo, " intentos."
FinSubProceso

// =====
// MÓDULO 4: JUEGO PIEDRA, PAPEL O TIJERA
// =====
SubProceso juego_piedra_papel_tijera
Definir eleccion_u, eleccion_pc Como Entero

Escribir "--- JUEGO: PIEDRA, PAPEL O TIJERA ---"
Escribir "1. Piedra"
Escribir "2. Papel"
Escribir "3. Tijera"
Escribir "Selecciona tu jugada (1-3):"
Leer eleccion_u

eleccion_pc <- Aleatorio(1, 3)
Escribir "La computadora eligio la opcion: ", eleccion_pc

Si eleccion_u = eleccion_pc Entonces
Escribir "Resultado: Empate."
SiNo
Si (eleccion_u = 1 Y eleccion_pc = 3) O (eleccion_u = 2 Y eleccion_pc = 1) O (eleccion_u =
3 Y eleccion_pc = 2) Entonces
Escribir "Resultado: Ganaste tu!"
SiNo
Escribir "Resultado: Gano la computadora."
FinSi
FinSi
FinSubProceso

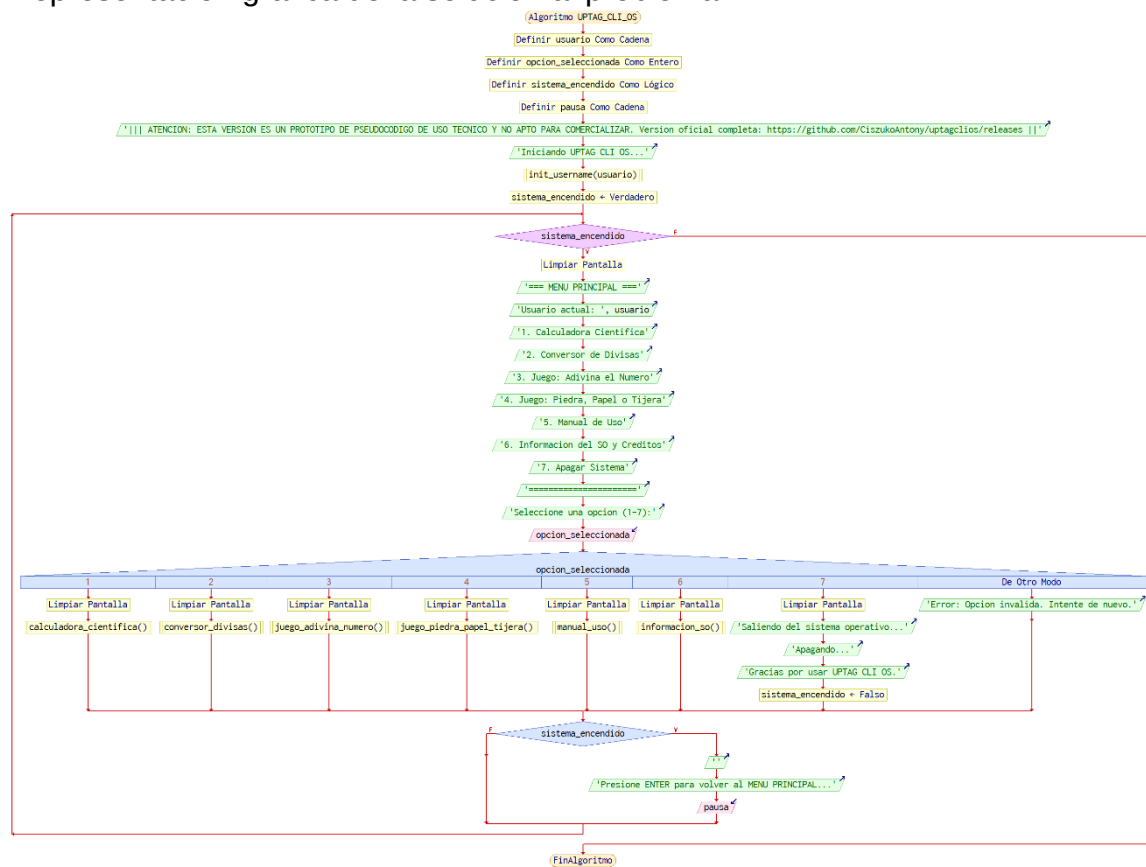
// =====
// MÓDULO 5: MANUAL DE USO
// =====
SubProceso manual_uso
Escribir "--- MANUAL DE USO ---"
Escribir "1. Menu Principal: Ingrese el numero entero de la opcion deseada."
Escribir "2. Modulos: Siga las instrucciones en pantalla ingresando datos coherentes."
Escribir "3. Salida Segura: Use la opcion 7 para cerrar el entorno sin corromper datos."
FinSubProceso

// =====
// MÓDULO 6: INFORMACIÓN DEL SO Y CRÉDITOS
// =====
SubProceso informacion_so
Escribir "--- INFORMACION DEL SO ---"
Escribir "Nombre: UPTAG CLI OS"
Escribir "Proyecto: Simulador Modular de Sistema Operativo Basado en Consola"
Escribir "Curso: Algoritmo y Programacion I"
Escribir ""
Escribir "DESARROLLADORES (EQUIPO):"
Escribir "- Francisco Garcia (C.I. 33.251.860)"
Escribir "- Luis Sanchez (C.I. 32.630.801)"
Escribir "- Ivan Quevedo (C.I. 34.570.333)"
Escribir "- Santiago Rojas (C.I. 33.652.368)"
Escribir "- Hanzer Sivira (C.I. 32.913.679)"
Escribir ""
Escribir "UPTAG - Coro, Falcon, Venezuela. 2026."
FinSubProceso

```

Diagrama de flujo

Representación gráfica de la solución al problema.



3. DESARROLLO (CODIFICACIÓN)

Trasladar el algoritmo a un lenguaje de programación para su ejecución. (main.py)

```
# Documentacion del main

"""
Modulo principal del sistema operativo.
---
Args:
    None.
---
Returns:
    None.
---
Raises:
    Exception: Si ocurre un error inesperado.
---
"""

# Documentacion del Mini Sistema Operativo UPTAG CLI OS

doc_uptag = """UPTAG CLI OS

MiniProyecto de Algoritmo y Programacion.
UPTAG PNfi S10 T1T1.
```


INTEGRANTES

- Francisco Garcia (33.251.860)
- Luis Sanchez (32.630.801)
- Ivan Quevedo (34.570.333)
- Santiago Rojas (33.652.368)
- Hanzer Sivira (32.913.679)

Coro, Falcon, Venezuela.
2026.

Descripción General

El proyecto consiste en el diseño e implementación de un simulador de Sistema Operativo basado en una Interfaz de Línea de Comandos (CLI). El sistema operará a través de un menú visual estructurado con bordes decorativos (ej. usando el carácter "=") e índices numéricos. Este entorno integrará un conjunto de utilidades prácticas y de entretenimiento, gestionadas a través de una arquitectura modular basada en funciones. El programa garantiza una experiencia de usuario continua gracias a un ciclo de ejecución infinito que solo se interrumpe cuando el usuario selecciona explícitamente la opción de salir, además de estar blindado contra errores de digitación mediante el manejo de excepciones.

A. Entradas (Inputs):

Navegación del Sistema: Selección de opciones a través de números enteros (del 1 al máximo de opciones disponibles en el menú principal).

Datos de Aplicaciones Utilitarias: * Calculadora Científica: Ingreso de expresiones matemáticas o números y operadores.

Conversor de Divisas: Ingreso del monto a convertir y selección de la moneda de origen y destino.

Datos de Entretenimiento (Juegos):

Adivina el Número: Ingreso de intentos numéricos enteros.

Piedra, Papel o Tijera: Selección numérica o de texto (strings) representando la jugada del usuario.

Señales de interrupción: Teclas como [ENTER] para continuar o comandos de salida.

B. Proceso (Process):

Bucle Principal (Estructura Repetitiva): Un ciclo while True (o "Mientras" en PSeInt) que mantiene el OS en ejecución constante, refrescando la pantalla y mostrando el menú principal tras cada acción terminada.

Manejo y Validación de Errores: Uso de bloques try...except para capturar errores de tipo (ej. ingresar una letra cuando se pide un número para navegar el menú) y evitar el "crasheo" del sistema.

Enrutamiento Lógico (Condicionales): Estructuras if / elif / else (o "Según / Caso") que evalúan la entrada del usuario y hacen el llamado a la función correspondiente.

Ejecución Modular (Funciones): Cada "aplicación" del sistema operativo vive en su propia función.

Lógica del Juego Adivinanza: Se genera un número aleatorio; un ciclo while evalúa la entrada del usuario usando condicionales para determinar la distancia absoluta y devolver el estado de "Frio" o "Caliente".

Lógica de PPT: Uso de la librería random para la jugada de la máquina y condicionales lógicos combinados (and/or) para determinar victoria, derrota o empate.

Limpieza de pantalla: Ejecución de comandos del sistema anfitrión (cls o clear) para simular el refresco de pantalla del OS.

C. Salidas (Outputs):

Interfaz Gráfica de Texto (TUI): Menús interactivos delineados con separadores visuales (=====) que muestran las aplicaciones instaladas:

Calculadora Científica

Convertor de Divisas

Juego: Adivina el Número

Juego: Piedra, Papel o Tijera

Manual de Uso

Información del SO y Créditos

Salir

Resultados Computados: Valores numéricos con formato exacto o decimal (resultados matemáticos y conversiones monetarias).

Feedback de Usuario: Mensajes de estado como alertas de error ("Entrada inválida, intente de nuevo"), pistas interactivas de los juegos ("¡Estás muy caliente!") y notificaciones del sistema ("Saliendo del OS...").

Requisitos Técnicos y Metodológicos a cumplir:

Trabajo Colaborativo: Cada integrante del equipo desarrollará y será responsable de al menos un módulo o "aplicación" funcional del OS.

Modularidad: Uso obligatorio de funciones (def en Python / SubProceso en PSeInt) para encapsular la lógica de cada opción del menú.

Documentación: El código fuente contará con Docstrings al inicio de cada función explicando sus parámetros y retornos, así como comentarios de línea (#) para bloques lógicos complejos.

Diseño Previo: Se entregará el Pseudocódigo y Diagrama de Flujo del menú principal y sus ramificaciones elaborados en la herramienta PSeInt.

¿Qué problemática resuelve?

En el contexto actual de la informática, los usuarios a menudo dependen de múltiples aplicaciones pesadas y separadas (una app de calculadora, consultas de divisas en navegadores web, juegos casuales independientes) para realizar tareas cotidianas sencillas. Esto genera tres problemas principales:

Consumo de recursos: Mantener múltiples aplicaciones o pestañas del navegador abiertas consume excesiva memoria RAM y ciclos de procesamiento, lo cual es ineficiente, especialmente en equipos de bajos recursos o hardware antiguo.

Pérdida de foco y tiempo (Context Switching): Cambiar constantemente entre diferentes interfaces gráficas distrae al usuario y reduce la agilidad para resolver problemas rápidos (como un cálculo relámpago o una conversión rápida).

Fragmentación Educativa: Desde la perspectiva del estudiante de informática, comprender cómo pequeñas piezas de código se unen para formar un ecosistema de software grande suele ser abstracto y difícil de asimilar.

La Solución aportada por el proyecto:

Este Mini Sistema Operativo CLI resuelve estas problemáticas ofreciendo un entorno unificado, ultra-ligero y libre de distracciones. Al centralizar herramientas vitales (matemáticas, financieras) y opciones de esparcimiento (juegos) en una sola interfaz de texto, el programa consume una cantidad ínfima de recursos del sistema anfitrión, respondiendo de manera instantánea.

Además, blindo al usuario contra la frustración operativa mediante su sistema de validación de errores (try-except), garantizando que la herramienta no se cierre abruptamente ante una

equivocación humana. Finalmente, desde el punto de vista académico, resuelve el reto de la integración: demuestra en la práctica cómo la arquitectura modular (modularidad por funciones) permite a un grupo de programadores colaborar en un mismo ecosistema, construyendo un producto escalable donde cada "aplicación" es un engranaje de un sistema mayor

Según el profesor Carlos Noguera, para obtener la calificación máxima, se debe cumplir con los siguientes criterios:

- Modularidad: Uso obligatorio de funciones (def en Python / SubProceso en PSeInt) para encapsular la lógica de cada opción del menú.
- Documentación: El código fuente contará con Docstrings al inicio de cada función explicando sus parámetros y retornos, así como comentarios de línea (#) para bloques lógicos complejos.
- Diseño Previo: Se entregará el Pseudocódigo y Diagrama de Flujo del menú principal y sus ramificaciones elaborados en la herramienta PSeInt

Copyright (c) 2026 Ciszuko

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE."

Importacion de librerias y modulos.

```
import time
import os
import random
import math
import platform
import sys
import Convertidor_divisa
import func_comprobacion
import func_juego_piedra_papel_tijera
from utils import clear_window, loading, ansi_text, init_username, call_username,
main_error, module_error
```

Main.

```
def main():
    """
    Función principal del sistema operativo.
    -
    Funcion principal main del sistema operativo UPTAG CLI OS.
    1. Calculadora Científica
    2. Conversor de Divisas
    3. Juego: Adivina el Número
    4. Juego: Piedra, Papel o Tijera
    5. Manual de Uso
    6. Información del SO y Créditos
    7. Apagar Sistema Operativo

    Args:
        None.
    """
```



```
opciones = [
    "Calculadora Científica", #1
    "Convertor de Divisas", #2
    "Juego: Adivina el Número", #3
    "Juego: Piedra, Papel o Tijera", #4
    "Manual de Uso", #5
    "Información del SO y Créditos", #6
    "Apagar Sistema Operativo" #7
]

print(f"{ansi_text.WHITE}={ansi_text.RESET}" * 100)
print(f""{ansi_text.ORANGE}
```

```

        else:
            print(f"{ansi_text.RED}Error: Opción inválida, {call_username()}, intente
de nuevo. Seleccione una opción dentro del rango {ansi_text.ORANGE}1 a
{len(opciones)}{ansi_text.WHITE} opciones... {ansi_text.RESET}")

            except Exception as error:
                print(f"{ansi_text.RED}Error: {error},
{ansi_text.CYAN}{call_username()}{ansi_text.RESET}, intente de nuevo. Seleccione una opción
dentro del rango {ansi_text.ORANGE}1 a {len(opciones)}{ansi_text.WHITE} opciones...
{ansi_text.RESET}")

                loading(1)
                input(f"\n{ansi_text.WHITE}=== {call_username()}, presiona
{ansi_text.GREEN}ENTER{ansi_text.WHITE} para volver al {ansi_text.ORANGE}MENU
PRINCIPAL{ansi_text.RESET} ===\n")

                print(f"\n{ansi_text.WHITE};Adios {call_username()}!")
                loading(1)

### Comprobación de main ###
if __name__ == "__main__":
    try:
        clear_window()
        main()

        except KeyboardInterrupt as kd:
            print(f"\n{ansi_text.RED}Error: (Control + C) detectado de forma abrupta...
{k d}{ansi_text.RESET}")

            except Exception as error:
                print(f"\n{ansi_text.RED}Error fatal del sistema: {error}{ansi_text.RESET}")

                print(f"\n{ansi_text.RED}Saliendo del sistema operativo...{ansi_text.RESET}\n")
                loading(3, 1, "Apagando...")
                clear_window()
                print(f"{ansi_text.WHITE}Gracias por usar {ansi_text.ORANGE}UPTAG CLI
OS.{ansi_text.RESET}\n")
                loading(2, 1, "Cerrando...")
                sys.exit()

    else:
        clear_window()
        main_error(__name__, __file__, __package__, __doc__)

```